Engenharia de Software

19/2/2026

TP1

Velocidade e Qualidade com Estruturas de Dados e Algoritmos

Professor(a): Ricardo Pires Mesquita

Samuel Hermany
INSTITUTO INFNET

Link GitHub

https://github.com/faculdade-infnet/VI-1-velocidade-qualidade-com-algoritmos/tree/main/TP1

1. Questão

Para retornar os caracteres individuais da string que não são caracteres de espaço em branco usando list comprehension, criei uma função e, nela, faço as seguintes verificações:

- Percorro cada caractere do texto enviado e verifico se não é um caractere em branco;
- No final da iteração completa do texto informado, são armazenados na variável caracteres apenas aqueles que correspondem à condição acima.

2. Questão

3. Questão

Para encontrar o número 8 no array ordenado, [2, 4, 6, 8, 10, 12, 13] conforme solicitado na questão, são necessários 4 passos. Isso ocorre pois é uma busca linear simples e o array é percorrido sequencialmente elemento por elemento, até que o valor procurado seja encontrado. Como o número 8 procurado está na quarta posição, então são realizadas 4 comparações.

4. Questão

Para encontrar o número 8 no array ordenado, [2, 4, 6, 8, 10, 12, 13] conforme solicitado na questão anterior, é necessário 1 passo. Isso ocorre pois é uma busca binária, ou seja:

- Obtém-se o total de elementos(7) - 1 = 6 pois a lista inicia a contagem em 0;
- Encontramos o meio, (início + último elemento /2);
- Substituindo (0 + 6)/2 = 3, este é o primeiro índice a ser verificado;
- Observa-se que os índices e elementos são:
  [0]2, [1]4, [2]6, [3]8. Logo o índice 3 corresponde ao número procurado.

5. Questão

Obs. Segundo enunciado a função encontra o maior número **único** dentro de um array, mas tem uma eficiência de $O(n^2)$, entretanto a função não faz realmente isso, ela encontra somente o maior número, não distingue se esse número é duplicado. Como pede no enunciado para modificar para O(n), foi isso que fiz.

No arquivo Python consta os dois códigos $O(n^2)$ e O(n).

Explicação passo a passo:

- Iniciei armazenando o primeiro valor da lista para ter com o que comparar
- No **loop fo**r eu percorro a quantidade de elementos da lista -1 pois inicio a comparação do próximo elemento que está sendo lido com o maiorValor armazenado.
- Se o próximo valor do loop for maior que o valor armazenado na variável maiorValor, então atualiza ela para o valor de comparação.
- Esse processo de comparação ocorre para cada elemento do for uma vez apenas **O(n)**.

- Após o fim do loop é retornado na função a variável maiorValor que contém o maior valor encontrado dentro do array
- Após isso o valor é exibido em tela.

6. Questão

A)

No meu entendimento, conclui que o número de grãos devia ser dividido até chegar ao mínimo que é 1. Explicação passo a passo:

- Dentro da função no início declaro a variável quadrados com valor 1, pois o mínimo de quadrados que podem existir é 1;
- Em sequência tenho o loop while onde verifico se o número de grãos informado é maior que 1, pois enquanto a condição for atendida continuará fazendo a divisão por 2 dentro do loop;
- Divido o numeroGraos atual por 2 e armazeno na própria variável;
- Adiciono + 1 a variável que faz a contagem dos quadrados;
- Assim ao final do loop tenho o número para saber em qual quadrado serão colocados a quantidade de grãos informados.

B)**TODAS AS IMAGENS ABAIXO SÃO ORIUNDAS DO DOCUMENTO QUE FIZ NO NOTION.SO.**

Link: https://www.notion.so/TP1-Quest-es-com-C-culo-de-complexidade-2f6ab4aadb258115a462f8ff9f865847?source=copy_link

```
# Custo Unitário = 5
def tabuleiro(numeroGraos):
  quadrados = 1                    # [custo 1] atribuição

  while numeroGraos > 1:           # [custo 1] comparação
    numeroGraos = numeroGraos /2   # [custo 1] atribuição
    quadrados = quadrados + 1      # [custo 1] atribuição
  return quadrados

# Csuto loop = 4
while numeroGraos > 1:
  numeroGraos = numeroGraos /2   # [custo 2] divisão + atribuição
  quadrados = quadrados + 1      # [custo 2] divisão + atribuição

# custo externo = 1
quadrados = 1                    # [custo 1] atribuição
```

Algoritmo Linear padrão $f(n) = an + b$

1. Equanto (4 passos):
   - Algoritmo linear

$$an \Rightarrow 4n$$

2. Custo externo inical e final:

   1

3. Queremos provar que $f(n) = O(n)$ logo $f(n) \leq C.g(n)$

$$f(n) = 4n + 1$$
$$4n + 1 \leq C\,n$$

4. Escolhe se a função dominante

$$g(n) = n$$

5. Eliminar a constante, encontrando o ponto de partida $n_0 \geq 1$ ou $n_0 = 1$

$$f(n) = 4n + 1$$
$$4n + 1 \leq 4n + n$$
$$4n + 1 \leq 5n$$
$$1 \leq 5n - 4n$$
$$1 \leq n$$
$$n \geq 1$$
$$n_0 = 1$$

   - Trabalhando a desigualdade $4n + 1 \leq Cn$
   - Se for $n \geq 1$

6. Substtuição de 1 por n

$$4n + n = 5n$$
$$f(n) = 4n + 1 \leq 5n$$

   - Tomamos como exemplo $C = 5$ e $n_0 = 1$
   - Então temos que para todo n ≥ 1 temos $f(n) = O(n)$.
   - Resposta:
     A função percorre o while e realiza operações constantes em cada iteração. Logo, sua complexidade de tempo é $O(n)$.

7. QUESTÃO

A seguinte função aceita um array de strings e retorna um novo array que contém apenas as strings que começam com o caractere "a". Use a Notação Big O para descrever a complexidade de tempo da função:

TODAS AS IMAGENS ABAIXO SÃO ORIUNDAS DO DOCUMENTO QUE FIZ NO NOTION.SO.

Link: https://www.notion.so/TP1-Quest-es-com-C-culo-de-complexidade-2f6ab4aadb258115a462f8ff9f865847?source=copy_link

```
# Custo Unitário = 5
def selectAStrings (array):
  newArray = []                        # [custo 1] atribuição

    for i in range(len(array)):        # [custo 1] tamanho da lista
      if array[i].startswith("a"):     # [custo 2] obter índice + verificação
        newArray.append(array[i])      # [custo 1] atribuição
    return newArray

# loop interno = 3
for i in range(len(array)):
  if array[i].startswith("a"):         # [custo 2] obter índice + verificação
    newArray.append(array[i])          # [custo 1] atribuição
return newArray

# custo externo = 1
newArray = []                        # [custo 1] atribuição
```

Algoritmo Linear padrão $f(n) = an + b$

1. Equanto (3 passos):
   - Algoritmo linear

$$an \Rightarrow 3n$$

2. Custo externo inical e final:

$$1$$

3. Queremos provar que $f(n) = O(n)$ logo $f(n) \leq C.g(n)$

$$f(n) = 3n + 1$$
$$3n + 1 \leq C\,n$$

4. Escolhe se a função dominante

$$g(n) = n$$

5. Eliminar a constante, encontrando o ponto de partida $n_0 \geq 1$ ou $n_0 = 1$

$$f(n) = 3n + 1$$
$$3n + 1 \leq 3n + n$$
$$3n + 1 \leq 4n$$
$$1 \leq 4n - 3n$$
$$1 \leq n$$
$$n \geq 1$$
$$n_0 = 1$$

- Trabalhando a desigualdade $3n + 1 \leq Cn$
- Se for $n \geq 1$

6. Substtuição de 1 por n

$$3n + n = 4n$$
$$f(n) = 3n + 1 \leq 4n$$

- Tomamos como exemplo $C = 4$ e $n_0 = 1$
- Então temos que para todo n ≥ 1 temos $f(n) = O(n)$.
- Resposta:
  A função percorre todos os elementos do array uma única vez e realiza operações constantes em cada iteração. Logo, sua complexidade de tempo é $O(n)$.

8. Questão

Aqui utilizei duas formas de ordenação simultâneas a cada iteração do loop while é realizado duas varreduras, uma à esquerda para a direita e outra da direita para a esquerda.

Com isso o número de passadas pela função tende a diminuir (pela metade aproximadamente), pois a cada passada a aplicação efetua a ordenação de 2 elementos, um mais à esquerda(início) e outra mais à direita(fim).

Entretanto é somente isso que muda, pois a complexidade continua a mesma, pois a cada passada ainda é necessário fazer duas comparações.

Entre os quatro tipos de lista que testei e pesquise, a diferença é perceptível onde os valores extremos estão fora do lugar, devido a quantidade de trocas ser menor.

9. QUESTÃO

Aqui a lista é composta por valores numéricos e é ordenada em ordem crescente.

O loop for externo percorre todos os itens da lista, já o loop for interno faz comparações entre elementos vizinhos. O if seguinte compara o valor atual com o próximo, se o elemento da posição atual for maior que o seguinte, ocorre a troca de posições.

Dentro do if, fazemos as trocas de posições e informamos em sequência que foi trocado pelo menos uma vez nessa passada.

O último if verifica se nenhuma troca foi realizada, caso isso ocorra, a lista já está ordenada e pode-se encerrar a aplicação antecipadamente.

10. QUESTÃO

Aqui a lista de strings é ordenada de forma crescente ignorando maiúsculas e minúsculas pois por padrão as maiúsculas sempre têm precedência na ordenação.

O loop for externo percorre todos os itens da lista, já o loop for interno faz comparações entre elementos vizinhos. O if seguinte compara a palavra atual com a próxima usando **.lower()** para ignorar diferenças entre maiúsculas e minúsculas.

Dentro do if, fazemos as trocas de posições e informamos em sequência que foi trocado pelo menos uma vez nessa passada.

O último if verifica se nenhuma troca foi realizada, caso isso ocorra, a lista já está ordenada e pode-se encerrar a aplicação antecipadamente.